

ĐƯỜNG ĐI NGẮN NHẤT {1}

A. MỤC TIÊU

- Sử dụng danh sách kề để cài đặt thuật toán Dijkstra.
- Sử dụng ma trận trọng số để cài đặt thuật toán Floyd.
- Giải quyết một số bài toán liên quan đến đường đi ngắn nhất.

B. BÀI TẬP PHẢI LÀM

Bài 1. Đường đi ngắn nhất (Dijkstra.*)

Cho đơn đồ thị có hướng có trọng số không âm $G=(V, E)$ và 2 đỉnh s, t . Hãy tìm đường đi ngắn nhất từ đỉnh s đến đỉnh t theo thuật toán Dijkstra.

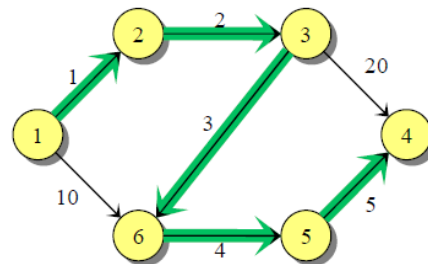
Dữ liệu:

- Dòng đầu tiên chứa số đỉnh n ($n \leq 1000$), số cung m ($m \leq 10^6$) của đồ thị và 2 đỉnh x, y .
- m dòng tiếp theo, mỗi dòng chứa 3 số u, v, w cho biết cung (u, v) là một cung có trọng số w (w là số nguyên có giá trị $|w| \leq 10^5$).

Kết quả:

- Dòng đầu tiên ghi số đỉnh đường đi ngắn nhất đi qua (tính luôn 2 đỉnh x và y) và độ dài đường đi.
- Dòng thứ 2 là danh sách các đỉnh của đường đi ngắn nhất tìm được từ s đến t .

Dữ liệu	Kết quả
6 7 1 4 1 2 1 1 6 10 2 3 2 3 4 20 3 6 3 6 5 4 5 4 5	6 15 1 2 3 6 5 4



Bài 2. Đường đi ngắn nhất qua đỉnh trung gian (NganNhatTrungGian.*)

Cho đơn đồ thị có hướng có trọng số không âm $G=(V, E)$ và 3 đỉnh s, x và t . Hãy tìm đường đi ngắn từ đỉnh s đến đỉnh t và đường đi đó phải đi qua đỉnh x .

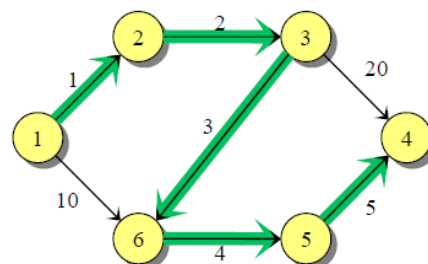
Dữ liệu:

- Dòng đầu tiên chứa số đỉnh n ($n \leq 1000$), số cung m ($m \leq 10^6$) của đồ thị và 3 đỉnh s, t , và x .
- m dòng tiếp theo, mỗi dòng chứa 3 số u, v, w cho biết cung (u, v) là một cung có trọng số w (w là số nguyên có giá trị $|w| \leq 10^5$).

Kết quả:

- Dòng đầu tiên ghi số đỉnh đường đi ngắn nhất đi từ đỉnh s đến t phải qua đỉnh x và độ dài đường đi.
- Dòng thứ 2 là danh sách các đỉnh của đường đi ngắn nhất tìm được từ s đến t .

Dữ liệu	Kết quả
6 7 1 4 3 1 2 1 1 6 10 2 3 2 3 4 20 3 6 3 6 5 4 5 4 5	6 15 1 2 3 6 5 4



Bài 3. Đường đi ngắn nhất mọi đỉnh (Floyd.*)

Cho đơn đồ thị vô hướng có trọng số không âm $G=(V, E)$. Hãy tìm đường đi ngắn nhất giữa các cặp đỉnh theo thuật toán Floyd.

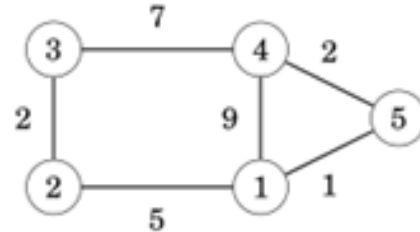
Dữ liệu:

- Dòng đầu tiên chứa số đỉnh n ($n \leq 1000$), số cung m ($m \leq 10^6$) của đồ thị.
- m dòng tiếp theo, mỗi dòng chứa 3 số u, v, w cho biết (u, v) là một cạnh có trọng số w (w là số nguyên có giá trị $|w| \leq 10^5$).

Kết quả: In ra n dòng và n cột ghi đường đi ngắn nhất giữa mọi cặp đỉnh.

Ví dụ:

Dữ liệu	Kết quả
5 6	0 5 7 3 1
1 2 5	5 0 2 8 6
1 4 9	7 2 0 7 8
1 5 1	3 8 7 0 2
2 3 2	1 6 8 2 0
3 4 7	
4 5 2	

**Bài 4. Dijkstra với hàng đợi ưu tiên (DijkstraPriorityQueue.*)**

Cho đơn đồ thị có hướng có trọng số không âm $G=(V, E)$ và 2 đỉnh s, t . Hãy tìm đường đi ngắn nhất từ đỉnh s đến đỉnh t theo thuật toán Dijkstra với hàng đợi ưu tiên (Có thể dùng cấu trúc dữ liệu *priority_queue* trong STL).

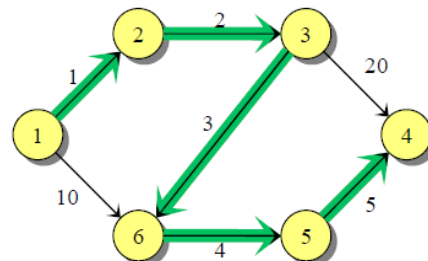
Dữ liệu:

- Dòng đầu tiên chứa số đỉnh n ($n \leq 1000$), số cung m ($m \leq 10^6$) của đồ thị và 3 đỉnh s, t , và x .
- m dòng tiếp theo, mỗi dòng chứa 3 số u, v, w cho biết cung (u, v) là một cung có trọng số w (w là số nguyên có giá trị $|w| \leq 10^5$).

Kết quả:

- Dòng đầu tiên ghi số đỉnh đường đi ngắn nhất đi qua (tính luôn 2 đỉnh s và t) và độ dài đường đi.
- Dòng thứ 2 là danh sách các đỉnh của đường đi ngắn nhất tìm được từ s đến t .

Dữ liệu	Kết quả
6 7 1 4	6 15
1 2 1	1 2 3 6 5 4
1 6 10	
2 3 2	
3 4 20	
3 6 3	
6 5 4	
5 4 5	

**HƯỚNG DẪN SỬ DỤNG MỘT SỐ LỚP/CẤU TRÚC KHÁC TRONG STL CÓ THỂ SỬ DỤNG TRONG DIJKSTRA****1. PAIR**

Pair là một cấu trúc gồm một cặp giá trị (first, second), trong đó first và second có thể có kiểu khác nhau.

Khai báo

```
#include <utility>
#include <string>
using namespace std;

pair<int, int> p1;
pair<string, int> p2;

// Thông thường để tiện lợi cho việc sử dụng, chúng ta đặt tên mới cho kiểu pair.
// Ví dụ typedef pair<int, int> ii;
```

Một số hàm

- Tạo cặp phần tử pair:

`p = make_pair(giá trị thứ 1, giá trị thứ 2);`

hay cũng có thể dùng:

`p = ii(giá trị thứ 1, giá trị thứ 2)`

- Truy cập các phần tử trong pair

`x = p.first;`

`y = p.second;`

Ví dụ

```
#include <utility>
#include <iostream>
using namespace std;

typedef pair<int, int> ii;

int main() {
    ii p;
    p = ii(1,5);
    cout << p.first << " " << p.second << endl;
    return 0;
}
```

2. PRIORITY_QUEUE

priority_queue là một hàng đợi ưu tiên, nghĩa là phần tử đầu tiên trong hàng đợi luôn luôn là phần tử lớn nhất trong các phần tử trong hàng đợi.

Khai báo

```
#include <queue>
using namespace std;

typedef pair<int, int> ii;

priority_queue<int> q1;
priority_queue < int, vector<int>, greater<int> > q2;
// Phần tử đầu tiên có giá trị nhỏ nhất
priority_queue < ii, vector<ii>, greater<ii> > q3;
```

Một số hàm thông dụng

- Thêm một phần tử: $O(1)$

`q.push(value);`

- Lấy giá trị ở đầu priority_queue (không xóa phần tử khỏi priority_queue): $O(1)$

`value = q.top();`

- Xóa 1 phần tử ở đầu priority_queue: $O(1)$

`void q.pop();`

- Lấy kích thước priority_queue: $O(1)$

`int q.size();`

- Kiểm tra priority_queue có rỗng không: $O(1)$

`bool q.empty();`

---o0o---

(Hết)